

Volatility Forecasting: A Statistical and Deep Learning Approach

Abstract

1 Introduction

1.1 Context

Modern share trading is often traced to Amsterdam in 1602, when the Dutch East India Company issued transferable shares and investors began trading them with one another [1]. At that point in time, it originated the possibility to earn money by buying the paper at a lower price, hoping that the company would grow with its expansion. Or, if someone had privileged information, e.g. a failed expedition or a successful one, would be in a better position than the others, being able to sell its position before it loses value or buy more before it gets a higher price.

The desire to obtain and interpret information that may affect stock prices remains central to financial markets. Services such as the Bloomberg Terminal give investors rapid access to news and data [2], while some traders use direct exchange feeds when even small delays matter [3]. If you receive the information before every player, you have a leverage over them and can position yourself in a beneficial way. Jim Simons, the founder of Renaissance Technologies, whose fund Medallion had an average annual return of 66% before investor fees, started his trading using news in early human trading decisions, before passing to models who studies historical market data as stated by Gregory Zuckerman in his Jim Simons biography, *The Man Who Solved the Market* [4].

As competition for information increases, the opportunity to profit from that same information decreases. Fama's efficient-market hypothesis states that prices reflect available information; in its weak form, this includes information in past prices [5]. An efficient market is one where prices adjust

quickly enough to available information such that an investor cannot reliably earn extra returns from it after accounting for risk and trading costs. Historical prices can still be studied for candidate patterns, but market efficiency gives no reason to assume that those patterns will yield profits. As described by Zuckerman, Simons searched historical price data for recurring patterns [4].

A historical data is known as a time series. Time series analysis has been a widely explored field in statistics and ML (Machine Learning). Statistical analysis is focused on understanding the properties of the data (stationarity, mean reversion, seasonality, trend, cycles, etc.) and creating a model, based on such characteristics that successfully explains the movements observed in such data. ML, in the meanwhile, is interested in creating a model that efficiently explains the data without effectively needing its traits.

1.1.1 Stastical Models

An early landmark of time series statistical modelling is Yule's 1927 autoregressive analysis, where the current value depends on past values [6]. In 1927, Slutsky demonstrated that summing random values can produce smooth, apparently cyclical movements [7], and in 1938, Wold showed that a weakly stationary time series can be split into a predictable component and another represented by current and past shocks [8]. Weak stationarity means a constant mean and finite variance, with covariance depending only on time lag. Both contributions served as foundations for moving-average representations.

Box and Jenkins's 1970 treatment popularized ARIMA (Autoregressive Integrated Moving Average) modeling, combining AR and MA terms with differencing [9]. In ARIMA(p, d, q), p and q correspond to the orders of AR and MA, respectively, while d is the number of differences used to address some forms of non-stationarity, especially stochastic trends. Their framework also treats seasonal ARIMA models. SARIMA(p, d, q)(P, D, Q)_s uses (p, d, q) for the nonseasonal component. Seasonality is a pattern that tends to recur every s observations; for monthly data with a yearly pattern, s = 12. P and Q are the orders of seasonal AR and MA parts, respectively, and D is the number of seasonal differences.

Box and Jenkins's models address the series' expected value over time. In a homoscedastic specification, the unexplained component is modeled as white noise ($\varepsilon_t \sim \text{WN}(0, \sigma^2)$) with constant variance. Robert Engle's 1982 ARCH (Autoregressive Conditional Heteroskedasticity) model instead studies time-varying conditional variance, originally in the context of uncertainty about future U.K. inflation [12]. A shock can have constant unconditional

variance while its variance conditional on past information changes over time. ARCH captures this by relating current conditional variance to past squared shocks.

Volatility often stays elevated for many periods, so ARCH can require many past squared shocks and consequently many parameters. Volatility is the standard deviation of returns, equal to the square root of their variance. Tim Bollerslev’s 1986 GARCH (Generalized Autoregressive Conditional Heteroskedasticity) model added lagged conditional variance terms to represent this persistence parsimoniously [13].

In 2009, Fulvio Corsi presented the HAR (Heterogeneous Autoregressive) model. Market participants act over different time horizons, so realized volatility measured over a day, a week, and a month may help predict future realized volatility [14]. Corsi drew on the heterogeneous-market hypothesis discussed by Ulrich A. Müller et al. in their 1993 working paper *Fractals and Intrinsic Time: A Challenge to Econometricians* [15].

AR, MA, ARIMA and SARIMA models the conditional mean. HAR presents a Realized-volatility regression. ARCH and GARCH the conditional variance. All shocks were modeled as WN. However, there are models who assume that shocks behaves like a Brownian Motion, mostly in econometry.

In 1900, Bachelier modeled speculative price changes using a Brownian-motion precursor [16]. In 1959, Osborne studied Brownian behavior in log price changes [17]. Samuelson’s 1965 warrant-pricing model used lognormal asset prices, corresponding to geometric Brownian motion [18]. In 1973, Fischer Black and Myron Scholes derived the theoretical price of a European call option on a non-dividend-paying stock under a geometric-Brownian-motion price model [19]. In 1998, Fabienne Comte and Eric Renault presented a continuous-time stochastic-volatility model with long memory [20]. Gatheral, Jaisson, and Rosenbaum’s 2014 preprint, published in 2018, proposed rough fractional stochastic volatility with a Hurst exponent below one half [21].

1.1.2 ML Models

In Machine Learning, the main idea is to create a model able to learn patterns from data. Neural networks combine weighted inputs through nonlinear layers, and their parameters can be adjusted by backpropagation. Werbos described the method in his 1974 dissertation [22]; Rumelhart, Hinton, and Williams demonstrated its use for learning internal representations in multi-layer networks in 1986 [23].

In 1987, Lapedes and Farber studied feedforward neural networks for nonlinear signal prediction and system modeling [29]. Waibel et al. used a time-delay neural network (TDNN) for phoneme recognition in 1989, apply-

ing shared computations across successive time positions [30]. Bai, Kolter, and Koltun evaluated a generic temporal convolutional network (TCN) with causal, dilated convolutions in 2018 [31]. Reisenhofer et al. developed HAR-Net in 2022, using causal convolutional layers that can be initialized to reproduce a fitted HAR forecast before optimization [32].

Elman’s 1990 recurrent neural network illustrates how a model can process sequential observations while carrying a hidden state forward [33]. Standard RNNs can have difficulty learning long-range dependencies because gradients may vanish or explode. Hochreiter and Schmidhuber introduced the LSTM (Long Short-Term Memory) in 1997 to help preserve error signals over longer spans [34]. Cho et al. introduced the gated recurrent unit (GRU) in 2014, using a simpler gated state than the LSTM; its parameter count and training speed depend on the chosen architecture and implementation [35].

In 2001, Breiman presented random forests, which aggregate predictions from randomized decision trees; for regression, their outputs are averaged [36]. Chen and Guestrin introduced XGBoost in 2016 as a scalable implementation of gradient-boosted trees, fitted sequentially to improve an objective function [37].

Vaswani et al. introduced the Transformer in 2017. Its self-attention relates positions in a sequence without a recurrent pass; autoregressive forecasting additionally requires a causal attention mask [38].

Each of these models have been used to predict returns or volatility even if they are not explicitly built for that reason as a HAR model is, for example, and that’s the main reason they are useful. They are not limited to one scope. One of the main benefits of ML models is generality.

1.1.3 Estimating Volatility

Unlike prices and returns, volatility is latent. It describes the dispersion of possible returns, so a single observed return cannot reveal its value: a single return reflects both the prevailing volatility and the particular random shock that occurred. In a continuous-time model, instantaneous volatility may also change within a trading day. Consequently, neither a daily closing price nor one daily return identifies the variance accumulated over that day.

Daily variance must therefore be estimated from observable prices. With sufficiently frequent observations and under suitable assumptions, the sum of squared intraday log returns approximates the day’s integrated variance; this is known as realized variance [39]. When intraday data are unavailable, range-based estimators use information in daily prices: Parkinson uses the high and low [41], while Garman–Klass uses the open, high, low, and close [42]. All are estimates rather than exact observations, and their accuracy

depends on the available data and the assumptions behind each estimator.

1.2 Motivations

The main objective with this project is understanding the better approach for predicting the price of an asset from a volatility forecast. The potential capabilities of ML models in finance forecasting tasks are tremendous, although the stability and robustness of statistical models are extremely reliable.

Several approaches have been presented as comparing both types of models. However, our objective is to extend these comparisons to include, besides the baseline models (AR, SARIMA, GARCH, HAR, MLP, LSTM), HARnet and RFSV models.

Furthermore, we are interested in understanding the generalization capacity of such models. Our focus is to understand the difference in performance between a local (fitted by one equity data) and a global (fitted by several tickers) model.

1.3 Contributions

In this paper, we present the comparison between AR(1), SARIMA(1,0,1)(1,0,1,5), GARCH(1,1), HAR-20 (base HAR using averages from 1, 5 and 20 observations), HAR-80 (HAR-20 extended to 40 and 80 observation averages), RFSV, LSTM, SiLU-LSTM (tanh activation function changed by SiLU), HARnet-20, HARnet-80. Each model will be fitted locally, one model fitted per each of 5 top-volume tickers, and globally, across a dataset of 2.714 tickers from NYSE and NASDAQ currency exchanges.

1.4 Related Work

Corsi’s heterogeneous autoregressive (HAR) model uses averages of realized volatility over daily, weekly, and monthly horizons to approximate persistent volatility dynamics with a small number of coefficients. Its empirical fits use intraday data for the USD/CHF spot exchange rate (Reuters quotes, 1989–2003), S&P 500 index futures (1990–2007), and 30-year U.S. Treasury bond futures (1990–2003); the equity exposure is an index future, not individual stocks [14]. Audrino and Chassot compare HAR with machine-learning methods using five-minute returns from NYSE TAQ trades in 2015–2023 for 1,445 U.S. stocks matched to CRSP. Their full sample includes 27 Dow Jones Industrial Average constituents and 69 Nasdaq-100 constituents, but is not restricted to those indexes. They show that the HAR fitting window

and re-estimation frequency materially affect the comparison; in their setting, a carefully fitted HAR remains a strong benchmark [43]. Reisenhofer, Bayer, and Hautsch extend this baseline with HARNet, a hierarchy of dilated causal convolutions initialized to reproduce a fitted HAR forecast before further optimization. They fit and test both 20- and 80-observation variants on five-minute realized-variance series for the S&P 500, FTSE, and Dow Jones Industrial Average indexes from the Oxford-Man Institute’s Realized Library, covering 2001–2020. These are index series rather than individual equities [32].

The choice between fitting one model per asset and pooling assets is another central question. Montero-Manso and Hyndman define a *local* method as fitting a separate forecasting function to each series and a *global* method as fitting one function using observations pooled across series. They show that a sufficiently flexible global method can represent the same forecasts as local methods even when the series differ. With a fixed finite input window, however, one global function must give the same forecast to identical inputs, whereas two local functions can differ; longer histories or series identifiers can distinguish such cases. Pooling increases the number of training examples available for shared parameters, but this is a modeling opportunity, not a guarantee that a global fit will forecast better. Their experiments use non-equity benchmarks including M1–M4 competition series, ATM cash demand, Wikipedia traffic, weather, and electricity demand, so their results motivate the local-versus-global design rather than directly establish its performance for stocks [44]. Liu et al. use 2014–2023 daily closing prices from Reuters Refinitiv Workspace for 11,771 individual stocks from exchanges in Asia, Europe, North America, and elsewhere; 10,240 series are available for global training and 1,531 stocks are reserved for unseen-stock evaluation. They report benefits from larger and more heterogeneous training pools [45]. Zhang et al. use Nasdaq ITCH order-book data supplied by LOBSTER for 93 S&P 500 constituent stocks selected from the 100 most liquid, spanning July 2011 to June 2021. They compare stock-specific and pooled models for intraday realized volatility, add a market-volatility proxy, and test previously unseen stocks [46]. These studies motivate evaluating local and pooled models separately, while recognizing that intraday predictors are not available for every asset in the present equity dataset.

Rough-volatility studies provide a complementary statistical approach. Gatheral, Jaisson, and Rosenbaum estimate roughness from DAX equity-index and German Bund bond futures using QuantHouse intraday data (2010–2014), and from S&P and Nasdaq index realized-variance series in the Oxford-Man library (2000–2014). They find evidence that log-volatility is rough and propose a rough fractional stochastic-volatility model with a

Hurst exponent below one half; this is estimation and model calibration, not neural-network training [21]. Mouti investigates roughness using Bloomberg daily open, high, low, and close data from April 2005 to April 2015. The asset set includes U.S. and international equity indexes (S&P 100, S&P 400, IBEX 35, FTSE 100, Hang Seng, and others) and individual shares such as Google and Microsoft; the main illustrations use S&P 100 and IBEX 35. Mouti also compares rough-volatility forecasts with AR, HAR, and GARCH forecasts [47]. Tang, Rosenbaum, and Zhou compare a pooled LSTM with a parsimonious rough-volatility model using five-minute Binance prices for 213 cryptocurrencies quoted against USDT. Their training period is 2020–2021 and their test period covers the first half of 2022; no equities or equity indexes are in that sample [48].

Finally, Brini compares pretrained, zero-shot time-series foundation models with econometric realized-volatility benchmarks on the VOLARE evaluation set: 40 individual U.S. stocks from all 11 GICS sectors (2015–2026), five major foreign-exchange pairs, and five commodity or equity-index futures. VOLARE’s five-minute realized measures are derived from Kibot tick data. The 50 assets are used for evaluation and for rolling estimation of the econometric benchmarks, not for task-specific training of the zero-shot foundation models. Brini finds that conclusions depend on how errors are aggregated across assets and whether forecasts are recalibrated [49]. The present thesis investigates models fitted on its own daily variance proxies, with local and global equity fits and matched forecast observations as the basis for eventual comparison. No result from the cited papers establishes the ranking of those models on this dataset.

1.5 Outline

On Section 2, we’ll be explaining the concepts behind volatility prediction, starting from statistical theory until ML theory necessary to understand the paper. On Section 3, we’ll be describing each model and our framework for evaluation. On Section 4, we’ll focus on implementation and reproducibility. On Section 5, we present our results. On Section 6, conclusion and further work.

2 Background

2.1 Statistics Concepts

2.1.1 Moments

Let $Y(t)$ be a random variable observed over time, and let $\mathcal{F}(t-1)$ contain the information available before time t . An unconditional property describes the distribution of $Y(t)$ without using that history; a conditional property describes its distribution given $\mathcal{F}(t-1)$. The distinction for the mean is

$$\mu = E[Y(t)], \quad \mu(t) = E[Y(t) \mid \mathcal{F}(t-1)]. \quad (2.1)$$

A moment summarizes an aspect of a distribution: its first moment concerns location, its second central moment dispersion, and its third central moment asymmetry.

For n observations, the sample mean estimates the population mean:

$$\bar{y} = \frac{1}{n} \sum_{t=1}^n y(t). \quad (2.2)$$

An autoregressive model illustrates a conditional mean. With the model shock $\varepsilon(t)$ satisfying $E[\varepsilon(t) \mid \mathcal{F}(t-1)] = 0$,

$$Y(t) = c + \phi Y(t-1) + \varepsilon(t), \quad \mu(t) = c + \phi Y(t-1). \quad (2.3)$$

SARIMA extends autoregressive and moving-average modeling with ordinary and seasonal differencing; it likewise specifies a forecast of the conditional mean of the modeled series. Differencing must be reversed when expressing a forecast on the original scale [9]. A fitted residual is the observed value minus the fitted conditional mean,

$$r(t) = y(t) - \hat{\mu}(t). \quad (2.4)$$

Thus $r(t)$ is calculated after fitting from observed data, whereas $\varepsilon(t)$ denotes the model's unobserved error or shock; fitted residuals approximate shocks.

The population variance and its sample estimate are

$$\text{Var}(Y(t)) = E[(Y(t) - \mu)^2], \quad s^2 = \frac{1}{n-1} \sum_{t=1}^n (y(t) - \bar{y})^2. \quad (2.5)$$

The estimator s^2 is unbiased for independent, identically distributed observations with finite variance: across repeated samples, its expected value equals

the population variance. Dividing by $n - 1$ compensates for estimating μ with the same sample's mean; dividing by n would underestimate variance on average. With dependent time-series observations, s^2 remains descriptive, but that unbiasedness is not automatic.

Conditional variance describes uncertainty left after observing the past. For $\varepsilon(t) = Y(t) - \mu(t)$, its conditional mean is zero, so

$$v(t) = \text{Var}(Y(t) \mid \mathcal{F}(t-1)) = \text{E}[\varepsilon(t)^2 \mid \mathcal{F}(t-1)]. \quad (2.6)$$

ARCH models $v(t)$ from past squared shocks; GARCH also uses past conditional variance:

$$\begin{aligned} v(t) &= \omega + \alpha \varepsilon(t-1)^2 && \text{ARCH}(1), \\ v(t) &= \omega + \alpha \varepsilon(t-1)^2 + \beta v(t-1) && \text{GARCH}(1,1). \end{aligned} \quad (2.7)$$

In estimation, lagged fitted residuals $r(t-1)$ stand in for the unobserved shocks. With $\omega > 0$, $\alpha, \beta \geq 0$, and $\alpha + \beta < 1$ (taking $\beta = 0$ for ARCH), these models can have constant unconditional variance while conditional uncertainty changes [12, 13].

Covariance generalizes variance to two variables:

$$\text{Cov}(X, Y) = \text{E}[(X - \text{E}[X])(Y - \text{E}[Y])], \quad \text{Var}(X) = \text{Cov}(X, X). \quad (2.8)$$

Applied to $Y(t)$ and $Y(t-k)$, it measures linear dependence across a lag k . The standardized third central moment is skewness, when the variance is positive and the third moment exists:

$$\text{Skew}(Y(t)) = \frac{\text{E}[(Y(t) - \mu)^3]}{\text{Var}(Y(t))^{3/2}}. \quad (2.9)$$

Its sign indicates the direction of asymmetry.

Homoscedasticity means that error variance is constant across observations; conditional homoscedasticity specifically means the variance given past information stays fixed. Heteroscedasticity means it changes. Thus a homoscedastic model has

$$\text{Var}(\varepsilon(t) \mid \mathcal{F}(t-1)) = \sigma^2, \quad (2.10)$$

whereas ARCH and GARCH set this conditional variance to $v(t)$ from (2.7). Their changing conditional variance does not itself imply changing unconditional variance [12, 13].

2.1.2 Shock Dynamics

A shock is the part of an observation that the conditional-mean model does not predict. Zero-mean white noise has constant finite unconditional variance and zero covariance between distinct dates. White noise need not be Gaussian or independent. In a homoscedastic SARIMA model, the innovations are commonly modeled as white noise with constant conditional variance. In ARCH and GARCH, the shocks follow

$$\varepsilon(t) = \sqrt{v(t)}z(t), \quad (2.11)$$

where the standardized innovations $z(t)$ are commonly assumed independent with mean zero and variance one. The shocks can be uncorrelated with constant unconditional variance while $v(t)$ changes [9, 12, 13].

In continuous time, standard Brownian motion $W(t)$ starts at zero, has continuous sample paths, has independent increments, and has stationary Gaussian increments:

$$W(0) = 0, \quad W(t) - W(s) \sim N(0, t - s), \quad t > s. \quad (2.12)$$

The increment variance is Δt , while its standard deviation is $(\Delta t)^{1/2}$. Although its increments are stationary, $W(t)$ itself is not:

$$\text{Var}(W(t)) = t. \quad (2.13)$$

White noise describes uncorrelated values at distinct dates; Brownian motion is a continuous-path process whose increments over disjoint intervals are independent.

Arithmetic Brownian motion adds drift and scale to that driver:

$$S(t) = S(0) + \mu t + \sigma W(t). \quad (2.14)$$

Its changes can be positive or negative without keeping $S(t)$ positive. Geometric Brownian motion instead models proportional changes:

$$dS(t) = \mu S(t) dt + \sigma S(t) dW(t). \quad (2.15)$$

To solve this equation, consider a general Brownian diffusion

$$dX(t) = a(t) dt + b(t) dW(t). \quad (2.16)$$

For a twice continuously differentiable, time-independent function f , the one-dimensional form of Itô's lemma states

$$df(X(t)) = \left[a(t)f'(X(t)) + \frac{1}{2}b(t)^2 f''(X(t)) \right] dt + b(t)f'(X(t)) dW(t). \quad (2.17)$$

Because $W(0) = 0$, a Brownian increment of length Δt has the exact scaling in distribution

$$W(\Delta t) \stackrel{d}{=} (\Delta t)^{1/2} Z, \quad Z \sim N(0, 1). \quad (2.18)$$

Thus its typical size is approximately $(\Delta t)^{1/2}$, and its square contributes at order Δt . Over uniform partitions of $[0, t]$, the squared increments have the nonzero limit

$$\sum_{i=0}^{n-1} \left[W\left(\frac{(i+1)t}{n}\right) - W\left(\frac{it}{n}\right) \right]^2 \xrightarrow[n \rightarrow \infty]{P} t. \quad (2.19)$$

This quadratic variation produces the second-derivative term in Itô's lemma. For $f(x) = \log x$ on $x > 0$,

$$f'(x) = \frac{1}{x}, \quad f''(x) = -\frac{1}{x^2}. \quad (2.20)$$

With $X(t) = S(t)$, $a(t) = \mu S(t)$ and $b(t) = \sigma S(t)$, Itô's lemma gives the extra $-\sigma^2/2$ term that an ordinary chain rule would miss:

$$d \log S(t) = (\mu - \sigma^2/2) dt + \sigma dW(t). \quad (2.21)$$

Integrating from zero to t , using $W(0) = 0$, gives

$$\log \frac{S(t)}{S(0)} = (\mu - \sigma^2/2)t + \sigma W(t). \quad (2.22)$$

Exponentiating gives a positive level for $S(0) > 0$, making geometric Brownian motion suitable for positive-valued quantities [16, 18, 19]:

$$S(t) = S(0) \exp\{(\mu - \sigma^2/2)t + \sigma W(t)\}. \quad (2.23)$$

Fractional Brownian motion $B^H(t)$ replaces the Brownian scaling exponent $1/2$ with a Hurst parameter $H \in (0, 1)$. Its centered Gaussian increments are stationary and satisfy

$$\begin{aligned} \text{Var}(B^H(t + \Delta t) - B^H(t)) &= (\Delta t)^{2H}, \\ \text{SD}(B^H(t + \Delta t) - B^H(t)) &= (\Delta t)^H. \end{aligned} \quad (2.24)$$

The case $H = 1/2$ recovers standard Brownian motion. For $H > 1/2$, fractional Brownian increments are positively correlated across time, indicating persistence and long memory in the increment process. For $H < 1/2$, the paths are rougher than Brownian paths: at increasingly fine time scales they remain continuous but look more jagged, because typical changes shrink more slowly as the time interval decreases [21].

Fractional Ornstein–Uhlenbeck (fOU) log volatility combines a mean-reverting drift with fractional Brownian forcing:

$$dX(t) = -\alpha(X(t) - m) dt + \nu dB^H(t), \quad \alpha > 0. \quad (2.25)$$

Its stationary form is

$$X(t) = m + \nu \int_{-\infty}^t e^{-\alpha(t-s)} dB^H(s). \quad (2.26)$$

Deviations from m decay at a rate set by α , while H controls local roughness [21].

2.1.3 Parameter estimation

Statistical fitting chooses model parameters from training observations by optimizing a specified criterion. For a linear conditional-mean model $y(t) = x(t)^\top \beta + \varepsilon(t)$, ordinary least squares (OLS) minimizes squared residuals:

$$\hat{\beta}_{\text{OLS}} \in \arg \min_{\beta} \sum_{t \in \mathcal{T}_{\text{train}}} (y(t) - x(t)^\top \beta)^2. \quad (2.27)$$

Here β is the vector of model parameters, and $x(t)$ contains the regressors available for the forecast, including an intercept if required. OLS fits the conditional mean of models such as AR and HAR; Gaussian errors are not needed to compute it. When errors are also independent Gaussian, OLS maximizes their likelihood conditional on the regressors.

Maximum likelihood instead chooses parameters that make the observed training series most probable under the assumed statistical model. With a conditional Gaussian mean $\mu_\theta(t)$ and positive conditional variance $\sigma_\theta^2(t)$, its log likelihood, up to initialization terms, is

$$\log L(\theta) = -\frac{1}{2} \sum_{t \in \mathcal{T}_{\text{train}}} \left[\log(2\pi\sigma_\theta^2(t)) + \frac{(y(t) - \mu_\theta(t))^2}{\sigma_\theta^2(t)} \right], \quad \theta^* \in \arg \max_{\theta} \log L(\theta). \quad (2.28)$$

The negative log likelihood defines a loss for a minimizing optimizer:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\log L(\theta), \quad \theta^* \in \arg \min_{\theta} \mathcal{L}_{\text{NLL}}(\theta). \quad (2.29)$$

The two objectives give the same θ^* : minimizing the negative log likelihood finds the parameters that best fit the observations under the assumed distribution. Unlike OLS with constant error variance, this criterion accounts for a variance that changes with the past, as in GARCH [13].

Rough fractional stochastic volatility (RFSV) instead estimates roughness from moments of log-volatility increments. Following Gatheral, Jaisson, and Rosenbaum's Section 2.1 notation, observe positive volatility on a grid $\sigma_0, \sigma_\Delta, \dots, \sigma_{N\Delta}$ over $[0, T]$, where $N = \lfloor T/\Delta \rfloor$. For moment order $q \geq 0$, their empirical moment is

$$m(q, \Delta) = \frac{1}{N} \sum_{k=1}^N |\log \sigma_{k\Delta} - \log \sigma_{(k-1)\Delta}|^q. \quad (2.30)$$

To interpret its slope and intercept, write W^H for the fractional Brownian driver B^H introduced above. Fractional Brownian increments obey

$$\mathbb{E}[|W^H(t + \Delta) - W^H(t)|^q] = K_q \Delta^{qH}, \quad K_q = \mathbb{E}[|Z|^q], \quad Z \sim \mathcal{N}(0, 1). \quad (2.31)$$

Thus K_q is the q th absolute moment of a standard Gaussian variable. Let $X(t) = \log \sigma(t)$. When the fOU mean-reversion rate is small over the lags considered, its increments approximately inherit the scaled fractional Brownian moments [21]:

$$\mathbb{E}[|X(t + \Delta) - X(t)|^q] \approx \nu^q K_q \Delta^{qH}, \quad m(q, \Delta) \approx \nu^q K_q \Delta^{qH}. \quad (2.32)$$

The second approximation treats the empirical moment as an estimate of the theoretical one. Taking logarithms for $q > 0$ and positive moments gives

$$\log m(q, \Delta) \approx q \log \nu + \log K_q + qH \log \Delta. \quad (2.33)$$

Consequently, in a regression across lags,

$$\log m(q, \Delta) \approx a_q + \zeta_q \log \Delta, \quad \zeta_q = qH, \quad a_q = \log(\nu^q K_q). \quad (2.34)$$

The slope ζ_q identifies roughness H , while the intercept a_q contains the increment scale together with K_q . This Gaussian-moment factor does not disappear for general q . Specifically, $K_2 = \mathbb{E}[Z^2] = 1$, so

$$a_2 = \log(\nu^2), \quad \nu^2 = e^{a_2}. \quad (2.35)$$

The fitted $q = 2$ intercept therefore estimates ν^2 as e^{a_2} , whereas the family of fitted slopes estimates H from the relation $\zeta_q = qH$ [21]. The purpose of this scale estimate appears in the paper's Section 5.2 variance predictor [21]:

$$\widehat{\sigma}_{t+\Delta}^2 = \exp\left\{\widehat{\log \sigma_{t+\Delta}^2} + 2c(H)\nu^2 \Delta^{2H}\right\}. \quad (2.36)$$

where

$$c(H) = \frac{\Gamma(3/2 - H)}{\Gamma(H + 1/2)\Gamma(2 - 2H)}. \quad (2.37)$$

Here $\widehat{\log \sigma_{t+\Delta}^2}$ is the conditional log-variance forecast from Section 5.1, and the $2c(H)\nu^2\Delta^{2H}$ term adjusts its exponent for conditional variance [21]. Thus H controls how increment size scales with lag and ν^2 its scale. The moment estimates use a volatility proxy because spot volatility is not directly observed; the proxy, sampling interval, and chosen lags can affect the estimates.

2.2 ML Concepts

2.2.1 Neural networks and layers

A neural network is a parameterized composition of layers that maps inputs to predictions. Let $x^{(0)} = x$ be its input vector. In a feedforward layer,

$$z^{(\ell)} = W^{(\ell)}x^{(\ell-1)} + b^{(\ell)}, \quad x^{(\ell)} = \phi(z^{(\ell)}). \quad (2.38)$$

The weights $W^{(\ell)}$ and biases $b^{(\ell)}$ are learned; the activation ϕ supplies non-linearity. Without nonlinear activations, a stack of such layers reduces to one affine map. The final layer maps the learned representation to the forecast, with its output transform chosen to match the target's units.

Common elementwise activations, for a scalar z , are

$$\begin{aligned} \text{sigmoid}(z) &= \frac{1}{1 + e^{-z}}, & \tanh(z) &= \frac{\sinh(z)}{\cosh(z)} = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \\ \text{ReLU}(z) &= \max(0, z), & \text{SiLU}(z) &= z \text{sigmoid}(z). \end{aligned} \quad (2.39)$$

Sigmoid maps to $(0, 1)$ and is useful for gates; tanh maps to $(-1, 1)$ and gives signed, bounded states. Both saturate at large $|z|$, yielding small derivatives there. ReLU has a zero derivative on negative inputs, so a unit that stays negative may stop learning. SiLU is smooth and retains a small nonzero response for negative inputs.

2.2.2 Losses, backpropagation, and optimization

A loss function $\ell(\hat{y}, y)$ assigns a numerical cost to predicting $\hat{y}$ when the observed value is y . Following the empirical-risk formulation of Reisenhofer, Bayer, and Hautsch [32], for a set $Z = \{(x_i, y_i)\}_{i=1}^N$ and model f_θ with parameters θ , the mean loss is

$$J(\theta; Z) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i). \quad (2.40)$$

For evaluation, write $\hat{y}_i = f_\theta(x_i)$ and $e_i = y_i - \hat{y}_i$. Let $s_i > 0$ be the naive scale associated with observation i . A one-step naive forecast simply repeats

the last observed value, so its mean absolute change over a reference history provides a scale for judging whether errors are large relative to that simple forecast. With these definitions, the five metrics are

$$\begin{aligned}
\text{MAE} &= \frac{1}{N} \sum_{i=1}^N |e_i|, & \text{MASE} &= \frac{1}{N} \sum_{i=1}^N \frac{|e_i|}{s_i}, \\
\text{MSE} &= \frac{1}{N} \sum_{i=1}^N e_i^2, & \text{RMSE} &= \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}, \\
\text{QLIKE} &= \frac{1}{N} \sum_{i=1}^N \left[\frac{y_i}{\hat{y}_i} - \log\left(\frac{y_i}{\hat{y}_i}\right) - 1 \right].
\end{aligned} \tag{2.41}$$

- **MAE** measures a typical absolute miss in the target’s units and is less sensitive to large errors than squared-error measures.
- **MASE** compares absolute errors with the naive scale and is dimensionless; a value below one means average scaled error below that reference scale, provided the scales are defined and positive [26].
- **MSE** squares errors, emphasizing large misses, and has squared target units.
- **RMSE** keeps the squared-error emphasis but returns to the target’s units.
- **QLIKE** is dimensionless and is zero for an exact prediction. Its logarithm requires $y_i > 0$ and $\hat{y}_i > 0$. For fixed $y_i > 0$, QLIKE diverges rapidly as $\hat{y}_i \rightarrow 0^+$, strongly penalizing underprediction, and grows without bound more slowly as $\hat{y}_i \rightarrow +\infty$. It has no real-valued limit as $\hat{y}_i \rightarrow -\infty$ because negative predictions are outside its domain [27].

An optimizer searches for parameters that minimize the chosen training loss, while evaluation can report several metrics on held-out observations. For training examples Z_{train} , the objective is

$$\theta^* \in \arg \min_{\theta} J(\theta; Z_{\text{train}}) = \arg \min_{\theta} \frac{1}{|Z_{\text{train}}|} \sum_{(x,y) \in Z_{\text{train}}} \ell(f_{\theta}(x), y). \tag{2.42}$$

The argmin is the set of parameter values attaining the smallest training loss; an optimizer aims to approach one of them. Backpropagation applies

the chain rule from the loss through each layer to compute $\nabla_{\theta} J$ [23]. Gradient descent then updates the parameters by

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} J(\theta_k; Z_k). \quad (2.43)$$

Here Z_k is the training batch for update k , and the learning rate $\eta > 0$ controls the size of each step in parameter space. Too large a step can overshoot a low-loss region; too small a step can make fitting slow. An optimizer specifies the update rule: ordinary stochastic gradient descent uses the current batch gradient, while Adam adapts each parameter’s step using running estimates of the gradient’s first and second moments [24]. Neither backpropagation nor an optimizer guarantees a global minimum.

2.2.3 Temporal convolutions

An ordinary one-dimensional convolution slides the same finite filter h along a sequence $x(t)$. If its supported offsets form a finite set S of integers, its output is

$$(x * h)(t) = \sum_{j \in S} h_j x(t - j). \quad (2.44)$$

The *receptive field* is the part of the input sequence that can affect one output. A filter with K consecutive weighted mappings covers K input positions in one layer; stacking layers can extend that field. Here K is the filter size, or the number of input-to-output mappings in the filter. An ordinary centered filter may use future observations. For forecasting, a *causal* convolution restricts these K mappings to the present and past:

$$(x *_c h)(t) = \sum_{j=0}^{K-1} h_j x(t - j). \quad (2.45)$$

A *dilated causal* convolution separates neighboring mappings by $d \geq 1$ observations:

$$(x *_c d h)(t) = \sum_{j=0}^{K-1} h_j x(t - jd). \quad (2.46)$$

Its K mappings span $1 + (K - 1)d$ positions without adding filter weights, although a single dilated layer uses only those K input positions. Bai et al. implemented causal filters with increasing dilations in their TCN architecture to let a convolutional network cover a long history without requiring an equally long filter [31]. HARNet uses hierarchies of such filters to represent multiple time horizons [32].

2.2.4 Model fitting

Fitting a forecasting model starts with a chronological division into training, validation, and test periods. Training is used for parameter optimization by means of gradient descent. Validation data guide choices such as architecture, learning rate, batch size, and when to stop training; the test period is held back for the final assessment. Using test outcomes to choose a model would bias that assessment. Any transform learned from data must likewise be fitted on training data alone.

These choices are *hyperparameters*: they are selected outside the parameter updates rather than learned as model weights. *Hidden size* counts the units in a hidden representation, an intermediate set of learned features passed to later layers. A wider representation can encode more features but increases model size and computation, with no guarantee of better test performance. Training examples are divided into *batches*, each used for one gradient update. Smaller batches give more frequent updates per pass through a fixed dataset, with noisier gradients; larger batches use more memory and fewer updates per pass. An *epoch* completes one pass through all training batches. The number of epochs controls how many passes are allowed, while validation loss can select a checkpoint or trigger early stopping.

2.3 Volatility Proxies

2.3.1 Latent volatility and integrated variance

Following Reisenhofer, Bayer, and Hautsch [32], let $p(t) = \log P(t)$ be the log price and suppose its continuous diffusion obeys

$$dp(t) = \mu(t) dt + \sigma(t) dW(t), \quad (2.47)$$

where $\mu(t)$ is the drift, $W(t)$ is Brownian motion, and $\sigma(t)$ is the latent spot volatility. Its squared value is instantaneous variance. The variance accumulated over day t is the integrated variance

$$IV(t) = \int_{t-1}^t \sigma^2(s) ds. \quad (2.48)$$

Neither $\sigma(t)$ nor $IV(t)$ is directly observed from a finite set of prices, so empirical work uses volatility proxies. Here IV means *integrated variance*.

2.3.2 Realized variance

Divide day t into N equally spaced intervals. The intraday log return for interval n is

$$r(t, n) = p\left(t - 1 + \frac{n}{N}\right) - p\left(t - 1 + \frac{n-1}{N}\right), \quad n = 1, \dots, N. \quad (2.49)$$

Realized variance (RV) is the sum of their squares:

$$\text{RV}_N(t) = \sum_{n=1}^N r(t, n)^2. \quad (2.50)$$

For the continuous diffusion above, with sufficiently fine, frictionless observations, $\text{RV}_N(t)$ is consistent for $\text{IV}(t)$ as $N \rightarrow \infty$ [40, 32]. Consistency means that, as the number of intraday observations in the same day increases, the estimator converges in probability to that day's integrated variance. Market microstructure noise and jumps can alter this limiting interpretation.

2.3.3 Parkinson estimator

When only daily high $H(t)$ and low $L(t)$ are available, Parkinson's range estimator gives the day's variance proxy

$$\hat{\sigma}_{\text{PK}}^2(t) = \frac{[\log(H(t)/L(t))]^2}{4 \log(2)}. \quad (2.51)$$

For a continuously observed, zero-drift Brownian log-price path with constant spot volatility $\sigma(t)$ over a unit-length day, $\text{IV}(t) = \sigma^2(t)$ and the squared log range satisfies [41]

$$\mathbb{E}[\{\log(H(t)/L(t))\}^2] = 4 \log(2) \sigma^2(t). \quad (2.52)$$

This expectation gives the denominator in (2.51). Discrete trading, drift, and jumps can affect its accuracy.

2.3.4 Garman–Klass estimator

Garman and Klass also use the daily open $O(t)$ and close $C(t)$. Their practical reduced estimator combines the squared log range with the squared open-to-close log return:

$$\hat{\sigma}_{\text{GK}}^2(t) = \frac{1}{2} \left[\log\left(\frac{H(t)}{L(t)}\right) \right]^2 - (2 \log 2 - 1) \left[\log\left(\frac{C(t)}{O(t)}\right) \right]^2. \quad (2.53)$$

Under the same continuous, zero-drift Brownian model, the open-to-close return has second moment $\sigma^2(t)$. Together with (2.52), this gives

$$\begin{aligned} \mathbb{E}[\{\log(C(t)/O(t))\}^2] &= \sigma^2(t) \\ \mathbb{E}[\hat{\sigma}_{\text{GK}}^2(t)] &= \frac{1}{2}[4\log(2)\sigma^2(t)] - (2\log(2) - 1)\sigma^2(t) = \sigma^2(t). \end{aligned} \tag{2.54}$$

Garman and Klass selected this practical form by dropping small cross-product terms from their more efficient quadratic estimator [42]. Like Parkinson's proxy, it uses prices within the observed session and can be biased by discrete trading or departures from the assumed price process.

2.4 Local and Global complexity

Following Montero-Manso and Hyndman, let $S = \{X_i \in \mathbb{R}^T : i = 1, \dots, K\}$ contain K observed series, and let each forecast cover H future observations. A local algorithm fits a separate forecasting function f_{θ_i} , with its own parameter vector θ_i , to each series. A global algorithm fits one function g_{θ} and one shared parameter vector θ across the set. For an observed series X_i , their forecasts are

$$\hat{Y}_i^{\text{local}} = f_{\theta_i}(X_i), \quad \hat{Y}_i^{\text{global}} = g_{\theta}(X_i), \quad i = 1, \dots, K. \tag{2.55}$$

Both forecasting functions map $\mathbb{R}^T$ to $\mathbb{R}^H$; the distinction is whether their fitted parameters are series-specific or shared [44].

To count fitted scalar parameters, define $P(f_{\theta}) = \dim(\theta)$. The total counts are

$$P_{\text{local}} = \sum_{i=1}^K P(f_{\theta_i}), \quad P_{\text{global}} = P(g_{\theta}). \tag{2.56}$$

Thus K local models with p parameters each use Kp fitted parameters, whereas a global model with q shared parameters uses q . The paper's formal complexity measure is $\mathcal{H}_i$, the hypothesis class, the set of all possible functions a model can learn. If $\mathcal{H}_i$ is the class for series i and $\mathcal{J}$ the global class, their complexities are

$$C_{\text{local}} = \prod_{i=1}^K |\mathcal{H}_i|, \quad C_{\text{global}} = |\mathcal{J}|. \tag{2.57}$$

The paper's relation in Section 3.4 places these classes on the same worst-case generalization bound when $C_{\text{local}} = C_{\text{global}}$, assuming equal in-sample error, independent series, equal effective sample sizes, and loss bounded in $[0, 1]$.

Parameter complexity is a proxy for this comparison only if each parameter has the same number B of possible values and distinct parameter choices give distinct functions. In that case, equal class cardinalities give

$$B^{P_{\text{local}}} = B^{P_{\text{global}}} \implies P_{\text{global}} = \sum_{i=1}^K P(f_{\theta_i}). \quad (2.58)$$

The paper illustrates this complexity with approximately $B = 2^{64}$ values per double-precision parameter in an autoregressive model with one parameter per lag [44].

2.5 Model Evaluation

Hansen, Lunde, and Nason's [28] model confidence set (MCS) compares a finite set of candidate models under one chosen loss function, without assuming that any candidate is the true data-generating model. Let $\mathcal{M}_0$ be the initial set, and let $L_i(t) = L(y(t), \hat{y}_i(t))$ be model i 's loss on a common evaluation observation at time t . Define the loss difference and its expected value by

$$d_{ij}(t) = L_i(t) - L_j(t), \quad \mu_{ij} = \mathbb{E}[d_{ij}(t)], \quad i, j \in \mathcal{M}_0. \quad (2.59)$$

A negative μ_{ij} favors model i . For a current candidate set $\mathcal{M} \subseteq \mathcal{M}_0$, Equation (1) of Hansen, Lunde, and Nason gives the equal-performance null; its alternative is that at least one pair differs:

$$\begin{aligned} H_{0,\mathcal{M}} : \quad & \mu_{ij} = 0 \quad \text{for every } i, j \in \mathcal{M}. \\ H_{1,\mathcal{M}} : \quad & \mu_{ij} \neq 0 \quad \text{for at least one } i, j \in \mathcal{M}. \end{aligned} \quad (2.60)$$

With $m = |\mathcal{M}|$ and n common evaluation times, define

$$\begin{aligned} \bar{d}_{i\cdot} &= \frac{1}{mn} \sum_{j \in \mathcal{M}} \sum_{t=1}^n d_{ij}(t), & t_{i\cdot} &= \frac{\bar{d}_{i\cdot}}{\sqrt{\widehat{\text{Var}}(\bar{d}_{i\cdot})}}, \\ T_{\max,\mathcal{M}} &= \max_{i \in \mathcal{M}} t_{i\cdot}, & e_{\max,\mathcal{M}} &= \arg \max_{i \in \mathcal{M}} t_{i\cdot}. \end{aligned} \quad (2.61)$$

A block bootstrap estimates the null distribution of $T_{\max, \mathcal{M}}$ and the variance of $\bar{d}_{i\cdot}$ from 10,000 block resamples drawn with replacement. For resample b , let $\bar{d}_{i\cdot}^*(b)$ be model i 's resampled mean excess loss. The centered bootstrap maximum is

$$T_{\max, \mathcal{M}}^*(b) = \max_{i \in \mathcal{M}} \frac{\bar{d}_{i\cdot}^*(b) - \bar{d}_{i\cdot}}{\sqrt{\widehat{\text{Var}}(\bar{d}_{i\cdot})}}. \quad (2.62)$$

Let $\Pr^*$ denote probability over these resamples. The upper-tail p-value is

$$p_{\mathcal{M}} = \Pr^*(T_{\max, \mathcal{M}}^* \geq T_{\max, \mathcal{M}}). \quad (2.63)$$

Starting with $\mathcal{M} = \mathcal{M}_0$, apply the following rule:

- If $p_{\mathcal{M}} < \alpha$, reject $H_{0, \mathcal{M}}$, remove $e_{\max, \mathcal{M}}$, and repeat with the smaller set.
- If $p_{\mathcal{M}} \geq \alpha$, stop and retain $\mathcal{M}$ as the $(1 - \alpha)$ MCS.

If only one model remains, retain it without another test [28].

3 Methods

3.1 Data

The equity stock universe is assembled from NASDAQ and NYSE listing files containing symbols, instrument names, and market capitalizations. These lists define candidate stocks, rather than constructed financial indices. For NASDAQ, symbols are stripped of surrounding whitespace, slashes are replaced by hyphens for Yahoo compatibility, and duplicates are removed. NYSE receives additional filtering: symbols containing a caret are excluded, and share classes are grouped by the base symbol obtained by removing a trailing class suffix. Within each group, the unsuffixed symbol is preferred, followed by class A; otherwise, the largest-market-capitalization row is selected. After this selection, instrument names containing Fund, Trust, Notes, Warrant, Preferred, Debentures, Senior Floating Rate, or Term are excluded without regard to case. Slashes are then replaced by hyphens and duplicates removed. The acquisition scheduler also removes duplicates from the combined exchange lists. The NYSE name and share-class rules are not applied to NASDAQ.

Each ticker is downloaded sequentially from Yahoo Finance through yfinance Python library, requesting its maximum available history with adjusted

OHLC prices and volume. We used a retry mechanism to handle rate limits and network errors.

The full dataset contains 22,817,463 observations across 5,842 tickers through September 10, 2026. The forecasting subset is smaller: 2,714 tickers have more than 80 raw observations before January 1, 2016 and a recorded observation on December 31, 2025. Training targets precede January 1, 2016; validation targets run from January 1, 2016 through December 31, 2018; test targets run from January 1, 2019 through December 31, 2025.

For local model estimation, five individual equities are selected from the 2,714-ticker cohort based on the highest mean daily trading volume before 2016, retaining only one share class per company: NVDA, AAPL, NFLX, GOOG, and AMZN. In equity markets, corporate share-split conventions maintain nominal share prices within comparable retail-accessible ranges, allowing raw trading volume to naturally capture prominent market leaders.

For the out-of-sample transfer evaluation on cryptocurrencies, five assets are selected from vendor history across the 2019–2025 period: BTC, ETH, SOL, XRP, and DOGE. Unlike equities, cryptocurrency token supplies span more than ten orders of magnitude without standardized denominations; selecting by raw token volume captures micro-priced tokens while omitting dominant market assets such as BTC (which ranks 113th in raw token volume). Assets are therefore selected by mean daily dollar volume ($\text{Close} \times \text{Volume}$), ensuring representative market liquidity.

The major observed variance proxy is the reduced Garman–Klass estimator in (2.53). We select it because it combines information from the intraday high-low range and the open-to-close log return, providing a daily variance proxy that uses all four OHLCV prices. Rows are invalid if OHLCV values are missing or nonfinite, prices are nonpositive, volume is negative, the high is below either open or close, the low is above either open or close, or the calculated variance is negative or nonfinite. Every input observation in a usable window must have valid, strictly positive variance. In the case of a zero variance: as an input, the window become invalid; as a target it becomes the floor value. The models need to understand the inputs that led to a zero target.

Furthermore, while equities rely solely on Garman–Klass (GK) estimator built from daily OHLC data, the cryptocurrency dataset builds both the daily Garman–Klass (GK) estimator in (2.53) and the 15-minute intraday realized variance (RV) proxy in (2.50) (summing 96 fifteen-minute squared log returns per UTC day), enabling transfer testing across both proxy formulations.

Let i identify a ticker and t its chronological observation number. Write $v_i(t) = \hat{\sigma}_{\text{GK},i}^2(t)$ for its original observed proxy, retaining the Background

estimator notation. A valid zero target receives the project adjustment

$$y_i(t) = \begin{cases} f_{\text{target}}, & v_i(t) = 0, \\ v_i(t), & v_i(t) > 0, \end{cases} \quad f_{\text{target}} = 3.396062419686545 \times 10^{-13}. \quad (3.1)$$

The floor is the smallest positive variance in the training cohort and is used because $\log(0)$ is undefined.

For a history length w , each forecast sample has the form

$$X_i(t) = (x_i(t - w + 1), \dots, x_i(t)), \quad Y_i(t + 1) = y_i(t + 1). \quad (3.2)$$

Here $x_i(t)$ is the input vector for that observation, formed from the variance proxy and, where used, the adjusted within-session log return. $X_i(t)$ represents the input sample, and $Y_i(t + 1)$ is the scalar next-observation target. Records are sorted by ticker and date, and histories never cross ticker boundaries. We consider Friday to Monday as one time step.

The saved coverage audit gives the following usable samples:

History length	Training	Validation	Test
20 observations	8,664,516	1,806,548	4,479,681
80 observations	7,004,011	1,692,154	4,295,773

3.2 Models

Each sample consists of one ticker's history and one next-observation target. Local models use their own parameter vector θ_i , whereas global models share one vector θ across these separate samples, as in (2.55).

Architectural scalar outputs are denoted $a(t + 1)$. Depending on the output convention, their conversion to variance is

$$\tilde{y}(t + 1) = \mathcal{T}(a(t + 1)), \quad \hat{y}(t + 1) = \max\{f_{\text{pred}}, \tilde{y}(t + 1)\}. \quad (3.3)$$

Here $\mathcal{T}(a) = a$ for a variance output, $\mathcal{T}(a) = e^a$ for log variance, and $\mathcal{T}(a) = e^{2a}$ for log volatility. The positive training-derived prediction floor f_{pred} is a lower limit, not an upper cap.

For AR and SARIMA, write $v(t)$ for the observed variance-proxy series and $\hat{v}(t + 1)$ for its conditional mean forecast, used as an estimate of conditional return variance. Their raw forecast in (3.3) is $\tilde{y}(t + 1) = \hat{v}(t + 1)$. Under the zero-conditional-mean innovation assumption, the future shock contributes zero to this forecast.

3.2.1 AR

The one-lag autoregressive baseline models variance by

$$v(t+1) = \beta_0 + \beta_1 v(t) + \varepsilon(t), \quad \widehat{v}(t+1) = \beta_0 + \beta_1 v(t). \quad (3.4)$$

Where $\varepsilon(t)$ is assumed to be $\text{WN}(0, \sigma^2)$. β_0 is an intercept and β_1 weights the latest variance [6].

3.2.2 SARIMA

Let L be the lag operator, $Lx(t) = x(t-1)$, for a generic series x . The seasonal autoregressive integrated moving-average specification for the variance series is

$$\phi(L)\Phi(L^s)(1-L)^d(1-L^s)^D v(t) = \vartheta(L)\Theta(L^s)\varepsilon(t). \quad (3.5)$$

The one-step variance forecast, conditional on the available within-ticker history $\mathcal{F}(t)$, is

$$\widehat{v}(t+1) = \mathbb{E}[v(t+1) \mid \mathcal{F}(t)]. \quad (3.6)$$

The polynomial definitions are

$$\begin{aligned} \phi(L) &= 1 - \sum_{j=1}^p \phi_j L^j, & \Phi(L^s) &= 1 - \sum_{j=1}^P \Phi_j L^{sj}, \\ \vartheta(L) &= 1 + \sum_{j=1}^q \vartheta_j L^j, & \Theta(L^s) &= 1 + \sum_{j=1}^Q \Theta_j L^{sj}. \end{aligned} \quad (3.7)$$

p, q are the nonseasonal autoregressive and moving-average orders; P, Q are their seasonal counterparts. d, D count ordinary and seasonal differences, and s is the seasonal period in observations. The coefficients $\phi_j, \Phi_j, \vartheta_j, \Theta_j$ weight past values and shocks; $\varepsilon(t)$ is a zero-mean white-noise innovation.

3.2.3 GARCH

GARCH uses the adjusted within-session return rather than variance as its driving observation:

$$\begin{aligned} r(t) &= \log\left(\frac{C(t)}{O(t)}\right), & \varepsilon(t) &= r(t) - \mu(t) = \sqrt{v(t)} z(t), \\ v(t) &= \text{Var}(r(t) \mid \mathcal{F}(t-1)), & z(t) &\sim \mathcal{N}(0, 1). \end{aligned} \quad (3.8)$$

$C(t), O(t)$ are adjusted close and open, $\mu(t)$ is a causal mean based on preceding within-ticker returns, and $\varepsilon(t)$ is the residual. $\mathcal{F}(t-1)$ contains the

information available before observation t , and $v(t)$ here is the conditional return variance, distinct from the observed variance proxy used as input to the other models. The GARCH(1,1) recursion and one-step variance forecast are

$$v(t) = \omega + \alpha \varepsilon^2(t-1) + \beta v(t-1), \quad \tilde{y}(t+1) = v(t+1). \quad (3.9)$$

The restrictions are $\omega > 0$, $\alpha, \beta \geq 0$, and $\alpha + \beta < 1$. ω supplies the base variance, α controls the response to squared shocks, and β carries variance persistence [13]. Volatility is $\sqrt{v(t+1)}$.

3.2.4 HAR

Define the trailing average over j observations by

$$\bar{v}^{(j)}(t) = \frac{1}{j} \sum_{k=0}^{j-1} v(t-k). \quad (3.10)$$

The heterogeneous autoregressive forecast is

$$\tilde{y}(t+1) = \beta_0 + \sum_{j \in \mathcal{J}} \beta_j \bar{v}^{(j)}(t), \quad \mathcal{J} = \{1, 5, 20\} \text{ or } \{1, 5, 20, 40, 80\}. \quad (3.11)$$

β_0 is the intercept and each β_j weights a distinct historical horizon. The first set defines HAR-20; the second extends it to HAR-80. Daily, approximately weekly, and approximately monthly averages give a parsimonious representation of dependence across horizons [14, 32]. The project uses a 20-observation monthly horizon in accordance with Reisenhofer, Bayer, and Hautsch [32], to ensure consistent window sizes when comparing HAR and HARNet.

3.2.5 RFSV

Rough fractional stochastic volatility uses the mean-reverting log-volatility dynamics:

$$X(t) = \log \sigma(t), \quad dX(t) = -\alpha(X(t) - m) dt + \nu dB^H(t). \quad (3.12)$$

Here B^H is fractional Brownian motion, where $0 < H < 1/2$ measures roughness, $\alpha > 0$ is the mean-reversion rate, m is the long-run log-volatility level, and ν scales the noise. $X(t)$ here is the scalar latent process. Gatheral et al.'s Equation (5.1) gives a historical kernel for log variance. With the positive proxy substituted for latent variance, write

$$\hat{\ell}(t + \Delta) = \frac{\cos(\pi H)}{\pi} \Delta^{H+1/2} \int_0^\infty \frac{\log v(t-u)}{(u+\Delta)u^{H+1/2}} du. \quad (3.13)$$

$\widehat{\ell}$ is the conditional log-variance predictor, u is backward lag, and $\Delta > 0$ is the forecast horizon. The variance mapping includes the Gaussian exponential-moment correction from the paper’s Section 5.2 predictor:

$$\widetilde{y}(t + \Delta) = \exp\{\widehat{\ell}(t + \Delta) + 2c(H)\nu^2\Delta^{2H}\}, \quad c(H) = \frac{\Gamma(3/2 - H)}{\Gamma(H + 1/2)\Gamma(2 - 2H)}. \quad (3.14)$$

Γ is the gamma function. H changes how past log variance is weighted. ν^2 scales the Gaussian correction in the variance forecast [21].

3.2.6 MLP

The multilayer perceptron flattens the input history into $x^{(0)}$ and applies three hidden layers:

$$\begin{aligned} z^{(\ell)} &= W^{(\ell)}x^{(\ell-1)} + b^{(\ell)}, & x^{(\ell)} &= \text{SiLU}(z^{(\ell)}), & \ell &= 1, 2, 3, \\ a(t+1) &= W^{(4)}x^{(3)} + b^{(4)}. \end{aligned} \quad (3.15)$$

The $W^{(\ell)}$ and $b^{(\ell)}$ are weight matrices and bias vectors, as in (2.38). SiLU is the elementwise activation in (2.39). The affine compositions mix information from all positions in the flattened history.

3.2.7 LSTM

For one position t in a chronological input sequence, concatenate the preceding hidden state and current input as $q(t) = [h(t-1); x(t)]$. Here $h(t)$ is a neural hidden-state vector, distinct from GARCH’s scalar conditional variance $v(t)$. The custom long short-term memory cell uses

$$\begin{aligned} i(t) &= \text{sigmoid}(W_i q(t) + b_i), & f(t) &= \text{sigmoid}(W_f q(t) + b_f), \\ g(t) &= \tanh(W_g q(t) + b_g), & o(t) &= \text{sigmoid}(W_o q(t) + b_o), \\ c(t) &= f(t) \odot c(t-1) + i(t) \odot g(t), & h(t) &= o(t) \odot \tanh(c(t)). \end{aligned} \quad (3.16)$$

i, f, o are input, forget, and output gates; g is the candidate update; c is the cell-state vector. $\odot$ means elementwise multiplication. The matrices W_i, W_f, W_g, W_o and vectors b_i, b_f, b_g, b_o parameterize the corresponding affine maps. The forget gate carries preceding cell information, the input gate admits new information, and the output gate controls its exposed representation [34]. After the last history position t , the scalar readout is

$$a(t+1) = W_{\text{out}} h(t) + b_{\text{out}}. \quad (3.17)$$

W_{out} is a row vector and b_{out} a scalar bias.

3.2.8 SiLU-LSTM

The sigmoid linear unit is

$$\text{SiLU}(z) = z \text{ sigmoid}(z) = \frac{z}{1 + e^{-z}}. \quad (3.18)$$

SiLU-LSTM preserves the sigmoid gates, cell update, and scalar readout of the preceding LSTM, replacing the candidate and cell-output hyperbolic tangents by

$$g(t) = \text{SiLU}(W_g q(t) + b_g), \quad h(t) = o(t) \odot \text{SiLU}(c(t)). \quad (3.19)$$

3.2.9 HARNet

The implemented HARNet uses a hierarchy of single-channel causal convolutions, following the nested-horizon design of Reisenhofer et al. [32]. Set $z_0(t) = v(t)$ and define each successive feature by

$$z_\ell(t) = \text{ReLU}\left(\sum_{k=0}^{K_\ell-1} a_{\ell,k} z_{\ell-1}(t - d_\ell k)\right), \quad \text{ReLU}(z) = \max\{0, z\}. \quad (3.20)$$

$a_{\ell,k}$ are filter weights, K_ℓ is filter length, and d_ℓ is dilation in observations. Convolutional biases are absent. The layer settings and resulting receptive fields R_ℓ are

$$\begin{aligned} (K_1, K_2, K_3, K_4) &= (5, 4, 2, 2), \\ (d_1, d_2, d_3, d_4) &= (1, 5, 20, 40), \\ R_0 &= 1, \quad R_\ell = R_{\ell-1} + (K_\ell - 1)d_\ell. \end{aligned} \quad (3.21)$$

Thus the features cover 1, 5, 20, 40, and 80 observations. HARNet-20 uses layers 1 and 2, while HARNet-80 also uses layers 3 and 4. With $L = 2$ or $L = 4$, respectively, their readout is

$$a(t+1) = b_{\text{out}} + \sum_{\ell=0}^L w_\ell z_\ell(t). \quad (3.22)$$

w_ℓ are scalar readout weights and b_{out} is its intercept.

4 Implementation

4.1 Input data

The configurations below describe the experiments selected by the current evaluation scripts. Global fits pool the 2,714-ticker forecasting dataset; local fits estimate parameters separately for the five tickers with the highest

average daily trading volume before 2016: NVDA, AAPL, NFLX, GOOG, and AMZN. Each target is next-recorded-observation, daily, unannualized adjusted Garman–Klass variance, with the zero-target adjustment defined in (3.1).

- **AR:** uses as input the latest estimator variance. At each possible training target, AR checks the preceding 20 recorded observations. If all are valid and positive and the target is valid, it adds the latest variance–target pair to least squares.
- **SARIMA:** uses the variance observations as input. It divides each ticker’s training history into segments, with at least 21 observations, of consecutive valid, positive-variance observations. An invalid observation ends the segment and is excluded; a valid zero-variance observation is floored, included as the final observation, and ends the segment.
- **GARCH:** uses the within-session log returns. It uses the same training segment rules as SARIMA. Its conditional mean is an expanding average of preceding returns within that segment.
- **HAR:** uses the raw variance histories. HAR-20 uses variance histories of 20 observations; HAR-80 uses 80. Their inputs are the variance averages for the corresponding horizons.
- **RFSV:** uses log variance history. Here T is the number of all preceding valid positive observations within the forecast ticker, excluding the target and future observations. Invalid and zero inputs are omitted from this full history, so T varies by ticker and forecast date.
- **MLP: Input:** the 20- and 80-observation variants use two features per observation: $\log \text{volatility} = \frac{1}{2} \log v(t)$ and $\log \text{return}$. The scalar output is $\log \text{variance}$, exponentiated for a variance forecast.
- **LSTM:** $\log \text{volatility}$ and $\log \text{return}$. The 20- and 80-observation variants process these features in chronological sequence order and output $\log \text{variance}$.
- **SiLU-LSTM: Input:** raw variance and $\log \text{return}$. The 20- and 80-observation variants produce a direct variance output.
- **HARNet: Input:** HARNet-20 and HARNet-80 receive variance alone, in 20- and 80-observation histories respectively, and produce direct variance outputs.

Training-derived positive prediction floors are applied after the output mapping in (3.3); each local fit derives its floor from its own ticker’s training history.

4.2 Hyperparameters

The neural model hyperparameters were selected by comparing baseline LSTM configurations with a fixed input window of 20 observations. Hidden sizes of $\{4, 16, 32, 64, 128\}$ were evaluated, with 128 yielding the best performance in these comparisons. This choice is also consistent with Zhang et al., who used 128 units in the first hidden layer of their LSTM architectures [46]. Learning rates of 10^{-3} and 10^{-4} were compared, and 10^{-3} was selected for its better convergence performance.

- **History length:** 20 and 80 observations for MLP, LSTM, SiLU-LSTM, HAR, and HARNet. AR uses one lag with window-20 eligibility; SARIMA and GARCH use preceding filtering history, window-20 scoring eligibility, and segment-based estimation. RFSV uses the full positive history T .
- **HAR averaging horizons:** HAR-20 uses $\{1, 5, 20\}$. HAR-80 uses $\{1, 5, 20, 40, 80\}$. HARNet uses the matching horizons.
- **AR order:** 1, with an intercept.
- **SARIMA orders:** nonseasonal $(1, 0, 1)$ and seasonal $(1, 0, 1, 5)$, with a five-observation seasonal period.
- **GARCH orders:** $(1, 1)$, with $\omega > 0$, $\alpha, \beta \geq 0$, and $\alpha + \beta < 1$.
- **Recurrent hidden width:** 128 for LSTM and SiLU-LSTM.
- **MLP hidden widths:** $128 \rightarrow 128 \rightarrow 2$, followed by one scalar output; all three hidden layers use SiLU.
- **HARNet hierarchy:** one channel per convolution, with filter lengths $(5, 4)$ and dilations $(1, 5)$ for HARNet-20; HARNet-80 adds lengths $(2, 2)$ and dilations $(20, 40)$. Convolutional biases are absent, and ReLU follows each convolution.
- **Adam learning rate:** initially 0.001 for the neural models. We use a learning rate scheduler.
- **Batch size:** 128 training samples.

- **Maximum epochs:** 20.
- **Patience:** 10 consecutive epochs without improvement in the configured validation loss.
- **Learning-rate step decay:** 0.1, applied once after the first patience interval.
- **Random seed:** 42 for neural initialization and batch shuffling.
- **RFSV moment orders:** $q \in \{1, 1.5, 2, 3, 4\}$ for log-volatility displacement moments.
- **RFSV observation lags:** 1–400 recorded-observation steps. H and ν^2 are fitted quantities, not chosen hyperparameters.

4.3 Fitting

All parameter estimation and initialization statistics use training observations only. Global statistical objectives aggregate within-ticker contributions without joining series across their boundaries; local objectives use only the selected ticker.

- **AR:** fit the intercept and one-lag coefficient by ordinary least squares (OLS).
- **SARIMA:** estimate the parameters by numerically maximizing the combined training-segment log likelihood using Powell optimization, by searching for parameter values that improve the objective, without calculating gradients. For each candidate parameter vector, the Kalman filter calculates each segment’s likelihood. It processes observations sequentially, predicting each value and updating the estimated state when the observation arrives. It also calculates the likelihood for each parameter set Powell tries.
- **GARCH:** subtracts the average of preceding returns from each training return. Parameters are estimated by numerically minimizing the combined Gaussian negative log likelihood using L-BFGS-B, with at most 100 iterations, by using gradients and an approximate curvature estimate, while respecting parameter bounds. Let s be the training residual mean square. Start from $\omega = 0.1s$, $\alpha = 0.1$, and $\beta = 0.8$. Each segment’s initial variance is its residual mean square, bounded below by $10^{-8}s$.

- **HAR:** estimate the intercept and horizon coefficients by OLS on the matching eligible 20- or 80-observation training samples.
- **RFSV:** adapt the moment method in *Volatility is rough* by Gatheral, Jaisson, and Rosenbaum [21] to pre-2016 Garman–Klass log volatility. Regress log displacement moments against log observation lag using log volatility from positive variance observations. Fit $\zeta(q) = Hq$ through the origin using the moment slopes; estimate ν^2 by exponentiating the $q = 2$ log-moment intercept. Global moments pool within-ticker displacement pairs; local regressions use the ticker alone.
- **MLP:** use minibatch Adam with variance-unit QLIKE. Initialize the output weights with Xavier uniform initialization and multiply them by 0.01; set the output bias to the log of the median training-history variance after the zero-target adjustment.
- **LSTM:** use minibatch Adam with variance-unit QLIKE. Scale the default output-layer weights by 0.01 and set the output bias to the log of the median training-history variance after the zero-target adjustment.
- **SiLU-LSTM:** use minibatch Adam with variance-unit MSE on raw direct outputs during training. Scale the default output-layer weights by 0.01 and set the output bias to the median training-history variance after the zero-target adjustment. Training outputs are not floored to preserve gradient flow; validation predictions are floored before calculating validation MSE.
- **HARNet:** initialize from the matching training-only HAR fit, using averaging convolution filters and the HAR intercept and coefficients in the final layer, then optimize with minibatch Adam and variance-unit QLIKE. On positive inputs this initialization reproduces the corresponding HAR forecast before the shared prediction floor.

Training batches are shuffled, while observations within each input sequence retain chronological order. After every epoch, validation selects the best checkpoint by QLIKE for MLP, LSTM, and HARNet, or by MSE for SiLU-LSTM. Ten consecutive unimproved epochs trigger one tenfold learning-rate reduction and reset the patience counter; another ten consecutive unimproved epochs stop fitting.

QLIKE is the neural training objective except for the selected SiLU-LSTM configuration, which uses MSE on direct variance outputs. Log-variance outputs are mapped back to variance before interpreting the loss.

QLIKE, MSE, RMSE, MAE and MASE are the evaluation metrics for comparison.

Earlier SiLU-LSTM trials with QLIKE exhibited numerical instability, including rejected nonfinite or nonpositive validation forecasts. Elfwing, Uchibe, and Doya [25] demonstrated that $dSiLU(z)$ when large positive z , approaches 1, whereas $dtanh(z)$ approaches zero. Repeated recurrent transformations can therefore preserve sensitivity or, conditionally on learned weights and gate values, amplify it; amplification is not guaranteed. The successful SiLU configuration also changes the input from log volatility to variance and the output from log variance to direct variance, so its stability cannot be attributed solely to switching the loss.

4.4 Evaluation Setup

The saved evaluations weight forecast observations equally. With the loss $\ell(\hat{y}, y)$ and mean loss $J(\theta; Z)$ from (2.40), the global loss uses one shared parameter vector:

$$J_{\text{global}} = J(\theta; Z_{\text{global}}) = \frac{1}{N_{\text{global}}} \sum_{(x,y) \in Z_{\text{global}}} \ell(g_{\theta}(x), y). \quad (4.1)$$

Local scores weight each ticker's mean loss by its forecast count:

$$J_{\text{local}} = \frac{\sum_{i=1}^M N_i J(\theta_i; Z_i)}{\sum_{i=1}^M N_i} = \frac{1}{\sum_{i=1}^M N_i} \sum_{i=1}^M \sum_{(x,y) \in Z_i} \ell(f_{\theta_i}(x), y). \quad (4.2)$$

Here g_{θ} has shared parameters, while f_{θ_i} has ticker-specific parameters as in (2.55). Z_{global} and Z_i contain the eligible forecast observations, with counts N_{global} and N_i ; $M = 5$ for local evaluation.

The saved tables use a test-relative MASE scale: for each ticker and window, divide absolute forecast errors by the naive mean absolute variance change over exactly that ticker's eligible test target dates, using the preceding observed variance for each naive forecast. RFSV-full uses the final target's absolute change from its preceding observation.

5 Experiments Overview

5.1 Goals

- Identify the best generalizing model on our dataset

- Compare related work results
- Analyze residuals structure
- Understand the parameter complexity between same model class.

5.2 Setting

All of the fitting and evaluation process were done in a machine with 16GB RAM, NVIDIA RTX 2060 6GB and Ryzen 5 5600x.

5.3 Computational Evaluation

At least 70 hours and 52 minutes of cumulative run time across global model selection, final global fits and local fits.

Global model	Recorded elapsed time
MLP-20	2 h 33 min
Base LSTM-20	3 h 52 min
SiLU-LSTM-20	4 h 26 min
HARNet-20	2 h 36 min
AR(1)	59 s
HAR-20	1 min 6 s
GARCH(1,1)	3 min 25 s
SARIMA	3 h 27 min
MLP-80	1 h 58 min
Base LSTM-80	9 h 19 min
SiLU-LSTM-80	9 h 40 min
HARNet-80	2 h 29 min
HAR-80	1 min 5 s
RFSV-20	—
RFSV-80	—

Table 5.1: Recorded global fitting times.

The Base LSTM-20 and MLP-80 entries cover only their resumed runs. Earlier runs also took time, but their durations were not recorded. RFSV parameter fitting was not recorded.

The five-stock local comparison required 49 minutes and 5 seconds for all 70 local fits.

6 Technical Evaluation

6.0.1 Overall Result

The equity tables report saved one-step forecasts of daily, unannualized adjusted Garman–Klass variance; Table 6.3 instead uses daily vendor crypto OHLC without equity adjustment. Forecasts are mapped back from any model-specific output transformation. MAE and RMSE have variance units, MSE has squared variance units, and MASE and QLIKE are dimensionless. Smaller losses are better. The equity source dataset has 2,714 tickers.

On 6.1, it’s clear to observe that models who used longer periods per forecast performed better than their counterparts. MLP-80 and Base LSTM-80 presented the best results. MLP-80 had the lowest MAE, MASE and second lowest MSE (RMSE follows), demonstrating an overall capacity to reconstruct actual values, while avoiding larger errors which would bring MSE up and keeping a better performance compared to the naive mean absolute change between observations. Base LSTM-80 had the lowest QLIKE and second best MAE and MASE, translating to a model which behaved best accordingly to practical standards where predicting lower values are worse than higher values. SiLU-LSTM-80 had the best performance for MSE, which was expected due to its fitting process, however had the second worse QLIKE value due to its inherently forecasting instability.

Model	MAE ($\times 10^{-4}$)	MASE	MSE ($\times 10^{-5}$)	RMSE ($\times 10^{-3}$)	QLIKE
MLP-20	8.81883	0.915832	6.32857	7.95523	0.423184
Base LSTM-20	8.30355	0.856715	6.15008	7.84224	0.417868
SiLU-LSTM-20	8.71735	1.05824	6.14899	7.84155	2.00879e+06
HARNet-20	9.32531	0.907195	6.69447	8.18197	0.430829
AR(1)	10.0775	1.78421	6.54807	8.09201	0.826027
HAR-20	8.89705	1.08316	6.30626	7.9412	0.513684
GARCH(1,1)	8.72768	1.0035	6.40219	8.00137	0.544997
SARIMA	8.80366	0.854352	6.3119	7.94475	811815
RFSV-20	10.1714	1.06676	6.28555	7.92814	0.486142
MLP-80	7.73236	0.835773	5.82827	7.63431	0.402411
Base LSTM-80	7.79013	0.836285	5.89105	7.67532	0.387572
SiLU-LSTM-80	8.06617	0.957564	5.74185	7.5775	855319
HARNet-80	8.52407	0.873897	6.10383	7.8127	0.398811
HAR-80	8.41118	0.959635	5.87548	7.66517	0.436633
RFSV-80	9.48762	1.04249	5.85084	7.64908	0.444466

Table 6.1: Global saved losses on eligible 2019–2025 targets. Window 20 has 4,479,681 forecasts per row and window 80 has 4,295,773. Red, blue, and dark orange mark the smallest, second-smallest, and largest values, respectively, per column across all rows.

The full-period rows of Table 6.2 evaluate NVDA, AAPL, NFLX, GOOG, and AMZN on the same 8,800 ticker/date targets during 2019–2025, selected by mean volume. Among those rows, the model that minimized absolute errors and performed the best compared to the mean change between observations is SARIMA Local. Following the results from 6.1, MLP-80 Global presented the best result on MSE and QLIKE. HARNET-80 had the second best values on MAE and MASE, while RFSV-80 Local on MSE and Base LSTM-20 Global on QLIKE.

Model	MAE ($\times 10^{-4}$)	MASE	MSE ($\times 10^{-7}$)	RMSE ($\times 10^{-4}$)	QLIKE
MLP-20 Local	2.41654	0.998704	1.98164	4.451560	0.323319
MLP-20 Global	2.38653	0.995313	1.93084	4.394130	0.289298
MLP-80 Local	2.66653	1.10488	2.18086	4.669960	0.365705
MLP-80 Global	2.15998	0.896018	1.79261	4.233920	0.279815
Base LSTM-20 Local	2.16824	0.907022	1.98629	4.456790	0.297445
Base LSTM-20 Global	2.22459	0.928232	1.86576	4.319440	0.280657
Base LSTM-80 Local	2.15813	0.886869	1.95862	4.425630	0.294209
Base LSTM-80 Global	2.13585	0.892587	1.81628	4.261780	0.282732
SiLU-LSTM-20 Local	2.48928	1.08808	2.16745	4.655580	0.905984
SiLU-LSTM-20 Global	2.7325	1.20579	2.01112	4.484550	0.372279
SiLU-LSTM-80 Local	2.24768	0.947574	2.20495	4.695690	0.318492
SiLU-LSTM-80 Global	2.38984	1.02927	1.98601	4.456470	0.328143
HARNet-20 Local	2.11595	0.892743	1.83777	4.286930	0.28784
HARNet-20 Global	2.20169	0.918785	1.85456	4.306470	0.283658
HARNet-80 Local	2.09498	0.865972	1.81179	4.256510	0.281475
HARNet-80 Global	2.12793	0.885052	1.81083	4.255390	0.28347
AR(1) Local	3.77586	1.63276	2.67056	5.167750	0.509255
AR(1) Global	4.94533	2.39741	3.60441	6.003680	0.688002
HAR-20 Local	2.46479	1.03921	1.91405	4.374990	0.329296
HAR-20 Global	2.79246	1.24376	2.14673	4.633280	0.398557
HAR-80 Local	2.28968	0.957714	1.8662	4.319950	0.303838
HAR-80 Global	2.41723	1.03541	2.02989	4.505430	0.333513
GARCH(1,1) Local	2.39042	0.988583	2.36177	4.85980	0.347896
GARCH(1,1) Global	2.61477	1.13228	2.3858	4.884470	0.383185
SARIMA Local	1.98639	0.830068	1.87609	4.331380	0.311451
SARIMA Global	2.09643	0.870198	2.08865	4.570180	0.308186
RFSV-20 Local	2.21821	0.924491	1.84617	4.296710	0.299769
RFSV-20 Global	2.86563	1.18549	2.41805	4.917370	0.324429
RFSV-80 Local	2.19111	0.914082	1.80254	4.245630	0.297806
RFSV-80 Global	2.80188	1.15992	2.25784	4.751670	0.322797

Table 6.2: Five-stock Local and Global saved losses on 8,800 shared 2019–2025 targets per row. Red, blue, and dark orange mark the smallest, second-smallest, and largest values, respectively, per column across all rows.

6.0.2 Crypto Evaluation

The saved equity-trained Global fits were also evaluated without refitting on the five dollar-volume-selected cryptocurrencies (BTC, ETH, SOL, XRP, and DOGE). Table 6.3 uses the same 1,760 positive target dates per crypto at both input windows, or 8,800 ticker/date forecasts per row. Each forecast targets the next recorded vendor daily, unannualized Garman–Klass variance.

Base LSTM-20 has the smallest crypto MAE and MASE, SiLU-LSTM-80 the smallest MSE and RMSE, and Base LSTM-80 the smallest QLIKE under daily Garman–Klass variance. Compared to the equities Global comparison, we can’t affirm that a longer window is better. Furthermore, MLP-80 dropped from the top, having the 4th largest QLIKE score. However, RFSV-80 entered the leaderboard with the second lowest QLIKE score.

Model	MAE ($\times 10^{-3}$)	MASE	MSE ($\times 10^{-4}$)	RMSE ($\times 10^{-2}$)	QLIKE
MLP-20	2.61274	0.847837	1.695	1.30192	0.614059
Base LSTM-20	2.39879	0.783858	1.41237	1.18843	0.614025
SiLU-LSTM-20	2.44326	0.799724	1.37919	1.17439	0.617657
HARNet-20	2.90899	0.922646	1.49235	1.22162	0.659521
AR(1)	2.46139	0.806905	1.55304	1.24621	1.05053
HAR-20	2.64562	0.842656	1.54782	1.24411	0.697185
GARCH(1,1)	2.64072	0.837037	1.55859	1.24843	0.788059
SARIMA	2.85337	0.897503	1.57661	1.25563	0.6737
RFSV-20	3.01448	0.991099	1.49621	1.22319	0.639008
MLP-80	2.49191	0.803978	1.62809	1.27597	0.676947
Base LSTM-80	2.43334	0.789239	1.50499	1.22678	0.58724
SiLU-LSTM-80	2.46853	0.801377	1.28433	1.13328	0.652116
HARNet-80	2.82639	0.895116	1.44734	1.20305	0.627904
HAR-80	2.75896	0.872503	1.50735	1.22774	0.646062
RFSV-80	2.94822	0.971546	1.47642	1.21508	0.612174

Table 6.3: Equity-trained Global losses on five unseen cryptocurrencies, with 8,800 retained targets per row. Red, blue, and dark orange mark the smallest, second-smallest, and largest values, respectively, per column across all rows.

To evaluate cross-proxy transfer across estimator formulations, the same 15 saved equity-trained Global fits were also evaluated on the 15-minute intraday realized variance proxy in (2.50) across the identical 8,800 ticker/date crypto sample, feeding preceding valid RV observations into the models. Table 6.4 reports the transfer performance under realized variance.

The results demonstrate the inexistence of a trend. For 6.4, LSTM-20 achieves the lowest QLIKE and MASE. For MAE only, SiLU-LSTM-20 scored the best, whereas for MSE RFSV-80 was superior. Severe QLIKE values are caused by underprediction and are driven by y/f_{pred} term, where as f_{pred} goes to zero, the quantity explodes, as shown by MLP-20, SiLU-LSTM-20 (which had the comprowed tendency of underprediction), SARIMA and SiLU-LSTM-80.

Model	MAE ($\times 10^{-3}$)	MASE	MSE ($\times 10^{-2}$)	RMSE ($\times 10^{-1}$)	QLIKE
MLP-20	9.82651	1.021733	2.39216	1.54666	1.34×10^{10}
Base LSTM-20	7.15176	0.872923	1.17044	1.08187	0.392958
SiLU-LSTM-20	7.03521	0.881654	1.22007	1.10457	4.63×10^8
HARNet-20	8.15446	1.009689	0.99869	0.99934	0.425783
AR(1)	8.12136	0.953755	1.64053	1.28083	0.917592
HAR-20	8.15655	0.964427	1.24176	1.11434	0.477661
GARCH(1,1)	10.04076	1.063117	2.40041	1.54932	2.08809
SARIMA	10.05437	1.086017	1.31837	1.14820	5.21×10^7
RFSV-20	8.57228	1.142607	1.27549	1.12938	0.429004
MLP-80	9.81243	1.011889	2.38012	1.54276	84.6598
Base LSTM-80	8.08906	0.908659	1.79441	1.33956	0.422430
SiLU-LSTM-80	7.65359	0.898995	1.50799	1.22800	3.45×10^6
HARNet-80	8.96509	1.017411	1.04651	1.02299	0.430985
HAR-80	9.99029	1.051481	1.36866	1.16990	0.461774
RFSV-80	7.07342	1.065548	0.96381	0.98174	0.421988

Table 6.4: Equity-trained Global losses transferred to 15-minute intraday realized variance on five unseen cryptocurrencies, with 8,800 retained targets per row. Red, blue, and dark orange mark the smallest, second-smallest, and largest values, respectively, per column across all rows.

6.0.3 Model statistical significance

As described in Section 2.5, we compare all 30 distinct local and global candidates on the same 1,760 test observations per stock using QLIKE and a 90% model confidence set. We run the procedure separately with moving bootstrap blocks of 20 and 80 retained observations. Both comparisons include the 20- and 80-observation forecast variants.

Each stock produces its own final set. Table 6.5 reports its size under each block length and the models appearing in only one of those sets. The intersection across the five stock-level final sets is identical for *both* block lengths: Base LSTM-20 Global; Base LSTM-80 Global; Base LSTM-80 Local; HARNet-20 Global; HARNet-80 Global; HARNet-80 Local; MLP-20 Global; MLP-80 Global; RFSV-20 Local; and RFSV-80 Local. Retention means the procedure did not reject a model from the remaining set at the chosen level; it does not establish equal expected loss.

Stock	MCS-20	MCS-80	Difference in membership
AAPL	10	10	None
AMZN	15	15	None
GOOG	19	20	AR(1) Local in MCS-80 only
NFLX	17	17	None
NVDA	22	22	None

Table 6.5: Number of retained models per stock from the same 30 QLIKE candidates, using 20- or 80-observation bootstrap blocks. Both runs use 1,760 common forecast targets per stock, 10,000 bootstrap draws, and a 90% model confidence set.

6.0.4 Residual Structure

With the purpose of analyzing the structure of residuals and providing a further understanding of the prediction behaviour of each model, we will provide residuals histogram and timelines for each model type and their local/global variants present in the final MCS set.

Raw residuals are actual minus predicted variance; individual log residuals are $\log v(t) - \log \hat{v}(t)$. Positive values indicate underprediction and negative values overprediction. Raw daily timelines show the cross-ticker mean actual minus mean predicted variance. Log daily timelines instead show $\log(\bar{v}(t)) - \log(\widehat{\bar{v}}(t))$, not the mean of individual log residuals. Cross-ticker averaging can conceal offsetting errors. Each histogram has an all-residual panel and a central-99.5% panel trimmed at the 0.25% and 99.75% quantiles for display only; the daily mean timelines and saved losses use all observations. Existing axis scales and histogram bins differ, so compare numeric axes rather than apparent panel widths or peak heights.

6.0.5 Parameter Complexity

For the idealized comparison in [44], take $B = 2^{64}$ possible encodings per scalar parameter and $K = 2,714$ tickers. For a model with p fitted scalars, the parameter-combination budgets are

$$C_{\text{local}} = B^{2,714p}, \quad C_{\text{global}} = B^p. \quad (6.1)$$

The local column in Table 6.6 extrapolates the selected architecture to all 2,714 tickers; only five local tickers were actually fitted, whose corresponding budget would be B^{5p} . These are encoding budgets, not counts of admissible finite coefficients: stability constraints and invalid floating-point encodings reduce the usable combinations. They equal counts of distinct forecasting

functions only under the distinguishability assumption. Neural hidden-unit permutations, for example, can represent the same function with different parameter vectors [50]. Thus this comparison concerns parameter storage and the potential combination space, not proof of realized function complexity or forecast quality.

Model	p	Global	Local (extrapolated)
AR(1)	2	2^{128}	$2^{347,392}$
HAR-20	4	2^{256}	$2^{694,784}$
HAR-80	6	2^{384}	$2^{1,042,176}$
GARCH(1,1)	3	2^{192}	$2^{521,088}$
SARIMA	5	2^{320}	$2^{868,480}$
RFSV-full	2	2^{128}	$2^{347,392}$
MLP-20	22,021	$2^{1,409,344}$	$2^{3,824,959,616}$
MLP-80	37,381	$2^{2,392,384}$	$2^{6,492,930,176}$
Base LSTM-20 / 80	67,201	$2^{4,300,864}$	$2^{11,672,544,896}$
SiLU-LSTM-20 / 80	67,201	$2^{4,300,864}$	$2^{11,672,544,896}$
HARNet-20	13	2^{832}	$2^{2,258,048}$
HARNet-80	19	$2^{1,216}$	$2^{3,300,224}$

Table 6.6: Parameter-combination budgets for the selected model specifications, with $B = 2^{64}$ possible encodings per scalar parameter. The local column assumes one parameter vector per ticker across the full cohort.

Counts include fitted coefficients and neural weights and biases, excluding optimizer state, filtering state, fixed hyperparameters, and preprocessing metadata.

7 Conclusions

References

- [1] Euronext. (2015). Gong marks exchange anniversary and exchange history website launch. Euronext Amsterdam.
- [2] Bloomberg Professional Services. (n.d.). Bloomberg Terminal: Real-time data, news, and analytics.
- [3] Nasdaq. (n.d.). Nasdaq TotalView: Direct market-data feed.
- [4] Zuckerman, G. (2019). *The man who solved the market: How Jim Simons launched the quant revolution*. Portfolio.
- [5] Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2), 383–417.
- [6] Yule, G. U. (1927). On a method of investigating periodicities in disturbed series, with special reference to Wolfer’s sunspot numbers. *Philosophical Transactions of the Royal Society of London. Series A*, 226, 267–298.
- [7] Slutsky, E. (1927). The summation of random causes as a source of cyclic processes. *Problems of Economic Conditions*, 3(1), 34–64. English translation in *Econometrica*, 5(2), 105–146 (1937).
- [8] Wold, H. (1938). *A study in the analysis of stationary time series*. Almqvist & Wiksell.
- [9] Box, G. E. P., and Jenkins, G. M. (1970). *Time series analysis: Forecasting and control*. Holden-Day.
- [10] Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45.
- [11] The Pennsylvania State University. (n.d.). Linear regression, Gauss–Markov theorem. *STAT 508: Applied Data Mining and Statistical Learning*, Lesson 4.
- [12] Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007.

- [13] Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.
- [14] Corsi, F. (2009). A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics*, 7(2), 174–196.
- [15] Müller, U. A., Dacorogna, M. M., Davé, R. D., Pictet, O. V., Olsen, R. B., and Ward, J. R. (1993). *Fractals and intrinsic time: A challenge to econometricians*. Olsen & Associates working paper.
- [16] Bachelier, L. (1900). Théorie de la spéculation. *Annales scientifiques de l’École Normale Supérieure*, 17, 21–86.
- [17] Osborne, M. F. M. (1959). Brownian motion in the stock market. *Operations Research*, 7(2), 145–173.
- [18] Samuelson, P. A. (1965). Rational theory of warrant pricing. *Industrial Management Review*, 6(2), 13–31.
- [19] Black, F., and Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.
- [20] Comte, F., and Renault, E. (1998). Long memory in continuous-time stochastic volatility models. *Mathematical Finance*, 8(4), 291–323.
- [21] Gatheral, J., Jaisson, T., and Rosenbaum, M. (2018). Volatility is rough. *Quantitative Finance*, 18(6), 933–949.
- [22] Werbos, P. J. (1974). *Beyond regression: New tools for prediction and analysis in the behavioral sciences* [Doctoral dissertation, Harvard University].
- [23] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- [24] Kingma, D. P., and Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.
- [25] Elfving, S., Uchibe, E., and Doya, K. (2017). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. arXiv:1702.03118, version 3, Equation (10).
- [26] Hyndman, R. J., and Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688.

- [27] Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1), 246–256.
- [28] Hansen, P. R., Lunde, A., and Nason, J. M. (2011). The model confidence set. *Econometrica*, 79(2), 453–497. doi:10.3982/ECTA5771.
- [29] Lapedes, A., and Farber, R. (1987). *Nonlinear signal processing using neural networks: Prediction and system modelling* (Technical Report LA-UR-87-1662). Los Alamos National Laboratory.
- [30] Waibel, A., Hanazawa, T., Hinton, G., Shikano, K., and Lang, K. J. (1989). Phoneme recognition using time-delay neural networks. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 37(3), 328–339.
- [31] Bai, S., Kolter, J. Z., and Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv:1803.01271.
- [32] Reisenhofer, R., Bayer, X., and Hautsch, N. (2022). HARNet: A convolutional neural network for realized volatility forecasting. arXiv:2205.07719.
- [33] Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179–211.
- [34] Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [35] Cho, K., van Merriënboer, B., Gülçehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Proceedings of EMNLP*, 1724–1734.
- [36] Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.
- [37] Chen, T., and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD Conference*, 785–794.
- [38] Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [39] Andersen, T. G., Bollerslev, T., Diebold, F. X., and Labys, P. (2003). Modeling and forecasting realized volatility. *Econometrica*, 71(2), 579–625.

- [40] Andersen, T. G., and Bollerslev, T. (1998). Answering the skeptics: Yes, standard volatility models do provide accurate forecasts. *International Economic Review*, 39(4), 885–905.
- [41] Parkinson, M. (1980). The extreme value method for estimating the variance of the rate of return. *The Journal of Business*, 53(1), 61–65.
- [42] Garman, M. B., and Klass, M. J. (1980). On the estimation of security price volatilities from historical data. *The Journal of Business*, 53(1), 67–78.
- [43] Audrino, F., and Chassot, J. (2024). HARd to beat: The overlooked impact of rolling windows in the era of machine learning. arXiv:2406.08041.
- [44] Montero-Manso, P., and Hyndman, R. J. (2021). Principles and algorithms for forecasting groups of time series: Locality and globality. *International Journal of Forecasting*, 37(4), 1632–1653.
- [45] Liu, C., Tran, M.-N., Wang, C., Gerlach, R., and Kohn, R. (2023). Global neural networks and the data scaling effect in financial time series forecasting. arXiv:2309.02072.
- [46] Zhang, C., Zhang, Y., Cucuringu, M., and Qian, Z. (2022). Volatility forecasting with machine learning and intraday commonality. arXiv:2202.08962.
- [47] Mouti, S. (2023). Rough volatility: Evidence from range volatility estimators. arXiv:2312.01426.
- [48] Tang, S. H., Rosenbaum, M., and Zhou, C. (2023). Forecasting volatility with machine learning and rough volatility: Example from the crypto-winter. arXiv:2311.04727.
- [49] Brini, A. (2026). Forecasting realized volatility with time series foundation models: A comparison with econometric benchmarks. arXiv:2607.05291.
- [50] Zhou, A., Yang, K., Burns, K., Cardace, A., Jiang, Y., Sokota, S., Kolter, J. Z., and Finn, C. (2023). Permutation equivariant neural functionals. *Advances in Neural Information Processing Systems*, 36. doi:10.52202/075280-1085.